

COMP4348/5348: Enterprise Scale Software Architecture

Semester 2, 2025

Lecture 8. Predicting performance and queues

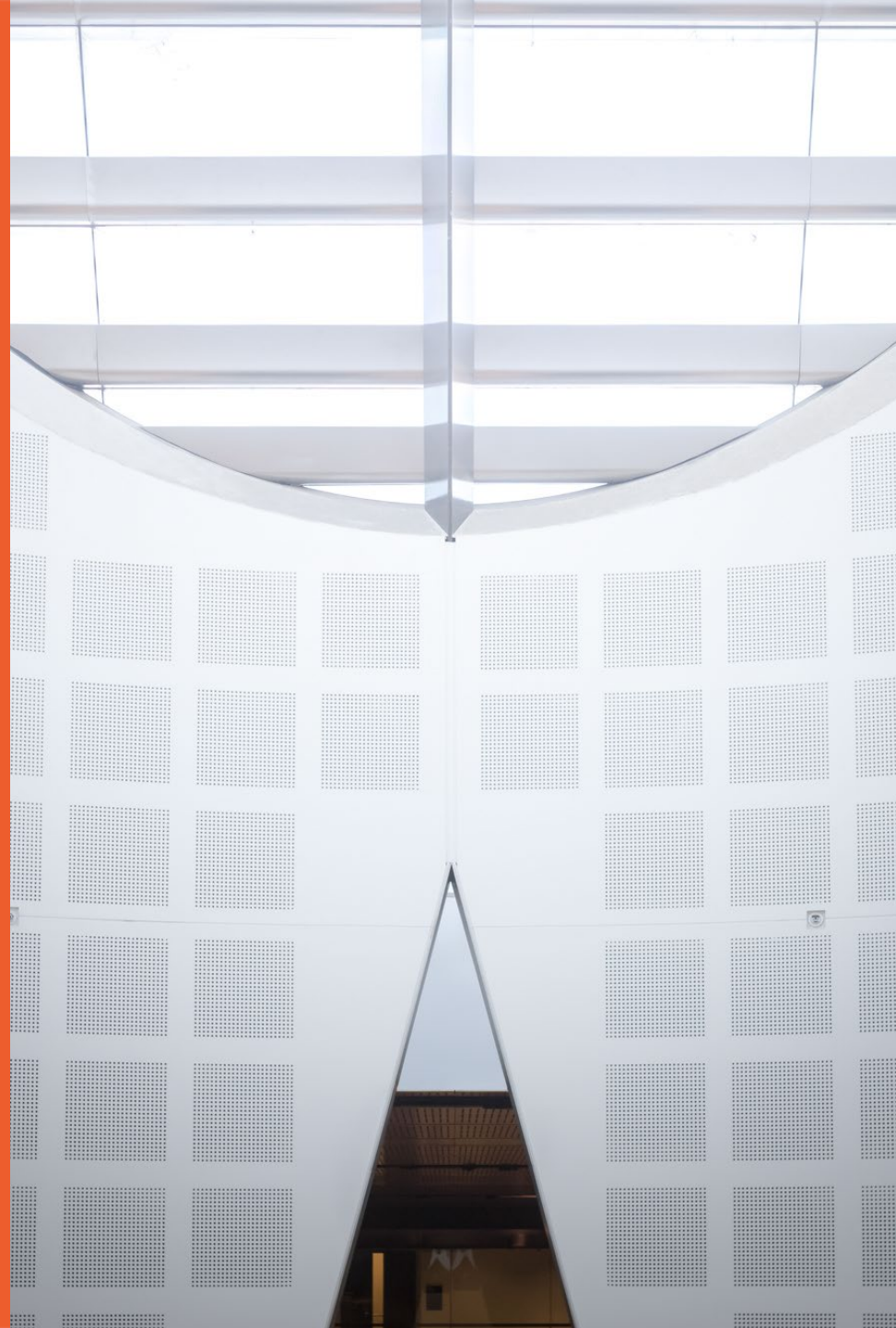
Dr. Liming Ge

School of Computer Science



THE UNIVERSITY OF
SYDNEY

Based on material by Basem Suleiman, Alan Fekete, Paul Greenfield,
Uwe Roehm, Matthew Sladescu and from
textbook by Gorton.



Copyright warning

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Sydney pursuant to Part VB of the Copyright Act 1968 (**the Act**).

The material in this communication may be subject to copyright under the Act. Any further copying or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice.

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

Extrapolation

- › Measure the performance for some values of vital parameters
 - Eg measure response time with 10, 20, 30 clients
- › Fit a curve through the measured data
- › Use the curve to predict performance for other values of parameters
- › Big problem: performance can change radically when system reaches capacity
 - Interpolation works well, but not extrapolation

Simulation

- › Build a computer-based model with tasks and resources and events
 - Based on randomized values for what is done, and how much resource each task uses
 - Using simulation frameworks eg NS2
- › Execute the model, and see what happens
 - Eg do queues build up? How many requests are handled in given time?
- › Problem: coding a reasonable model is very difficult, and takes a lot of time
- › Problem: getting key parameters is hard
 - Eg how long each request uses a given disk

Analytical (Queuing) Models

- › Build a mathematical model, with resources and queues and probabilities for state transitions
- › Solve for the steady state behaviour
 - Either exact solution (as a function)
 - Or approximate solution of the relevant equations by iteration
- › Not as precise as simulation, but usually far easier provided the pattern (distribution) of random values is known to be one of a small number of easy-to-work-with forms
- › Some simple results can be done back-of-an-envelope, without depending on precise knowledge of the distribution of random values
 - called “operational laws” and the focus in this unit of study

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

Inequalities

- › If $A < B$ and $B < C$ then $A < C$
- › If $A < B$ then $A+X < B+X$
- › If $A < B$ and $C < D$ then $A+C < B+D$
- › If $A < B$ and $X > 0$ then $A*X < B*X$
- › If $0 < A < B$ then $1/B < 1/A$

Basic probability (discrete cases)

- › $P(A)$: the fraction of occurrences of event A among all possible cases
 - Eg cases are all possible ways to toss 20 coins
 - One case is HTHHTHTHTTHTTTTTHHHTH
 - A case is a simple event $P(\text{HTHHTHTHTTHTTTTTHHHTH}) = 1/2^{20}$
 - One event is “A1: fourth toss is heads”; $P(A1) = 0.5$
 - Another event is “A2: First head is at toss 4”; $P(A2) = 0.0625$
- › $P(A)$ must be between 0 and 1 (inclusive)
 - What does $P(A) = 0$ mean?
- › $P(\text{not } A) = 1 - P(A)$
- › $P(A \text{ and } B) = P(A) * P(B)$
 - If A independent of B (they are not correlated)
- › $P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$
 - Approximately $P(A) + P(B)$ if A independent of B, and both are rare

Random variable (discrete cases)

- › Give a numeric value to each possible situation
 - Eg cases are possible ways to toss 20 coins; Random value is number of heads seen
- › Expected value
 - Weighted average of random variable among the various cases
- › $E(X) = \sum_i X(\text{case } i) * P(\text{case } i)$

Uniform distribution (discrete cases)

- › Integer valued random variable
- › All values are equally likely, within the range allowed

Binomial distribution (discrete cases)

- › Number of successes among n attempts, when each attempt has chance p of success
- › $P(X=k) = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k}$
- › Expected value $E(X) = np$
- › See http://en.wikipedia.org/wiki/Binomial_distribution

Continuous cases

- › Where there are infinitely many possible situations
 - Eg each situation is one execution of a system
- › Assign probabilities to events
 - An event is a subset of situations
 - Eg event is "no processor crashed during the execution"
 - Or "the 5th request was processed at node 17"
 - Events can overlap with one another
- › Random variable is a value associated with each situation
 - Eg "the number of requests processed during the execution"
- › Probability density function

Uniform distribution

- › Discrete cases: All values are equally likely, with a finite set of allowed values
 - Continuous cases: Any range has probability proportional to the length of the range (as long as range is within the set of allowed values)
- › Eg “all file lengths are equally likely, from 0 bytes to 1 Mbyte”
- › Easy to work with mathematically, but may not be an accurate approximation to reality!

Normal distribution

- › Two parameters μ and σ
- › [Non-examinable formula] $\text{Pdf}(x) = (\sigma \sqrt{2\pi})^{-1} \exp(- (x - \mu)^2 / \sigma^2)$
- › Distribution is symmetric around $X = \mu$
 - Expected value is μ
 - σ is a measure of the spread
- › Many real situations are closely approximated as normal distributions
 - Combined impact of many independent variations
- › See http://en.wikipedia.org/wiki/Normal_distribution

Exponential distribution

- › One parameter λ
- › [Non-examinable formula] Pdf(x) = $\lambda \exp(-\lambda x)$
- › Expected value = $1/\lambda$
- › Memoryless distribution
 - Time between events, if the past has no impact on the future
- › See http://en.wikipedia.org/wiki/Exponential_distribution

Poisson distribution

- › Number of occurrences of an event with exponential distribution on inter-event gaps
 - A good approximation to binomial distribution when $np = \lambda$, and n is big and p is small
- › The probability of having value k is
 - [Non-examinable formula] $\text{Prob}(k) = (\lambda^k \exp(-\lambda))/k!$
- › Expected value = λ
- › See http://en.wikipedia.org/wiki/Poisson_distribution

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

The main independent parameter

› For a closed system

- N : the number of clients that submit jobs
 - Also, the number of jobs that are either in the system, or “thinking” at the client
- Z : think time, between when a job is completed and when the same client submits another job

› For an open system

- λ : the arrival rate of jobs per second

Work is done at different devices

- › V_i : (average) number of visits by a single job to device i
- › S_i : average service time (time spent actually using the device, not queueing etc) for a visit at device i
- › λ_i : average arrival rate of visits per second at device i
- › X_i : Throughput of device i (visits completed per second)
- › μ_i : average service rate of visits at device i
 - Number of visits that could be completed per second, if the device were kept continually busy
 - $= 1 / S_i$
- › D_i : service demand *per job* at device i
 - $= V_i * S_i$
- › U_i : Utilisation of device i
 - Fraction of time device is busy
 - $= \lambda_i * S_i$

The main performance measures

- › Throughput
- › X: the (average) number of jobs completed per second
- › Response time
- › R: the (average) time from submitting a job till it is completed

Little 's Law

- › $N = \lambda T$
 - Applies to any system in steady state
 - N is number of jobs in the “system”
 - T is the average time spent in the “system”
 - λ is the average rate of arrival (or departure) from the system
- › This law can be applied to any part of the system
 - It doesn't depend on knowing the shape of the distribution of parameters
- › It can be applied to a larger whole with a closed computer-system plus the clients
 - Thus N is the number of clients, because each client causes 1 job somewhere (thinking at the client, or being processed, or waiting somewhere)
 - $N = X^*(R+Z)$, equivalently:
 - $R = N/X - Z$
 - This is called “interactive response time law”

Relating a device to the whole system

- › Forced Flow Law
 - $X_i = V_i * X$
- › Service Demand Law (“Bottleneck Law”)
 - $U_i = D_i * X$

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

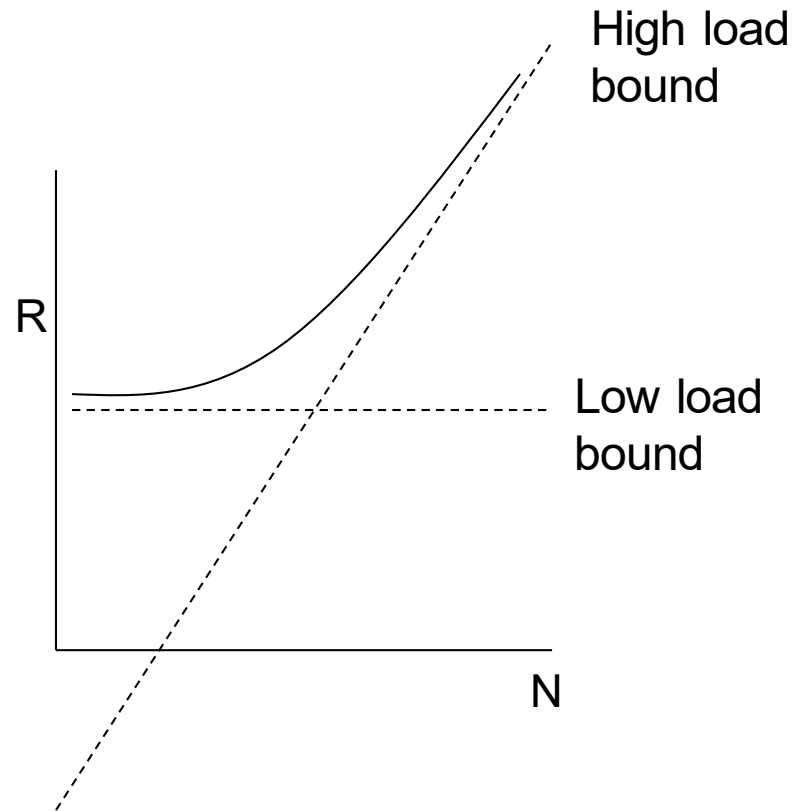
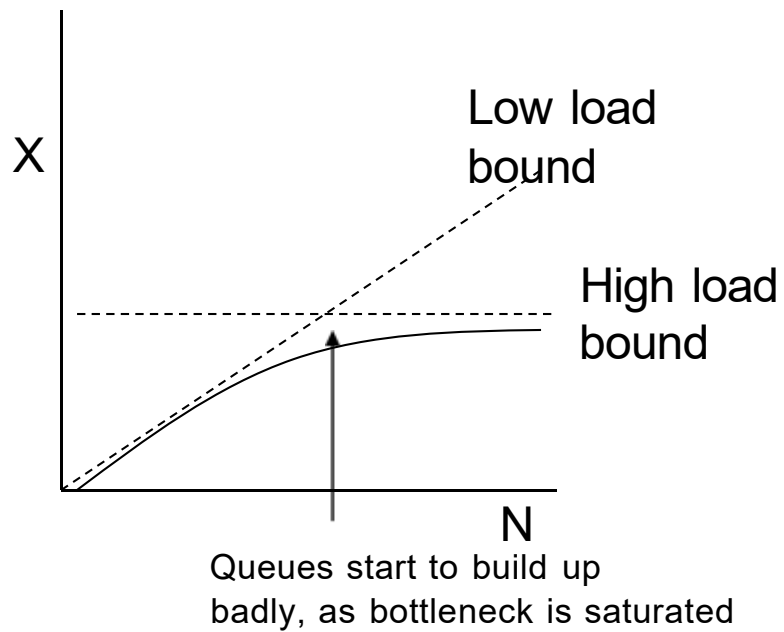
Performance bounds

- › Let $D_{\max}$ be the largest among the D_i
- › The device i where the max occurs is called the “bottleneck” for the system
- › $X \leq 1 / D_{\max}$
 - Because U_i must beat most 1.
- › So $R \geq N^* D_{\max} - Z$
- › X and R are close to the bounds, when load is high

Performance bounds

- › Let D be the sum of all the D_i
- › $R \geq D$
 - Because the total time to deal with a job includes the time at each device, plus also queuing time
- › So $X \leq N / (D + Z)$
- › X and R are close to the bounds, when load is low

Effect of the bounds



Planning for Scale

- › Typical situation: we have an existing system, with certain performance properties. We want to know how much the workload can increase, before the performance becomes unacceptable.
- › The question we may ask: how many clients can be supported, while keeping average response time below a given limit?”
 - Equivalently “how much throughput can be deliver, while keeping average response time below a given limit?”
 - Some variations are much harder to deal with: “while keeping 99% of response times below a limit”
- › From the existing system measurements, we can determine the categories of jobs, and the service demand D_i on each resource i
 - We assume that adding more clients, of the same type, doesn't alter D_i
- › The high load bounds can be solved to find the highest N that allows R to be below the limit required
 - then determine the X bound for this N

System redesign methodology

- › In any system, the bottleneck is the device i with largest D_i
- › The only way to get better throughput than $1/D_{\max}$ is to change the system design to change $D_{\max}$
 - Move some tasks away from the bottleneck device
 - Add some new devices to take some of the load
 - Or re-arrange processing so some visits go to other existing devices instead
 - Or, change the nature of the device so each request needs less service time
 - Eg change for a faster disk
- › After the system is redesigned, there will still be a bottleneck!
 - Perhaps on a different device

Warning

- › Common mistake is to re-use a previous calculation of a quantity which has changed in the new situation
 - eg from higher N , we have the same D_i but different X_i and U_i
 - eg from change in frequency of various job types, we have different (average) D_i

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

M/M/1

- › Sometimes we can assume jobs arrive by Poisson process
 - Inter-arrival time gap has exponential distribution
 - Average number of arrivals per second is λ
- › Also, assume the service demand is exponential distribution
 - Average number of services possible per second is μ
- › The essential parameter is load ρ , defined by λ/μ

M/M/1 Queuing Theory

- › Average number of jobs in (queue or being served) is $N = p/(1 - p)$
- › Average number of jobs in queue is N_{wait}
 - $N_{\text{wait}} = p^2/(1 - p)$
- › Average time in system (waiting or being served) is $T = 1/\mu(1 - p)$
- › Average waiting time in queue is $T_{\text{wait}} = p / \mu(1 - p)$
- › Note: proofs not needed, but know these results

Impact of queues

- › If λ is close to μ , then the queue builds up to be long
 - And each job is delayed a long time
 - Note contrast with situation of exactly periodic arrival of jobs, and exactly equal service times, where there is never a queue as long as λ is below μ
- › We aim to keep load p at the bottleneck resource to be below 0.8, say
 - Thus we do not let any resource get saturated

Heavy tailed distributions

- › In general, distributions which are spread more from the average have longer queues and worse response times
- › This is especially serious for distributions like power-laws, where the few extremely big jobs can clog the system giving bad response time for every job

Summary

- › There are several different approaches to performance prediction
 - analytical approach (calculate bounds by formulae from knowledge of workload and system features)
- › Typical analysis for closed system with N clients: Identify system resources where work from different jobs can be done concurrently
 - determine D_i for each resource; then find D and $D_{\max}$
 - use high-load (or sometimes also low-load) bounds to estimate response time and throughput
 - low-load bounds: $R \geq D$; $X \leq N / (D + Z)$
 - high-load bounds: $X \leq 1 / D_{\max}$; $R \geq N * D_{\max} - Z$
 - interactive response-time law: $R = N/X - Z$
 - identify the bottleneck resource
- › If you know there are exponential-like job arrival and task duration, estimate queue delays $T_{\text{wait}} = p / \mu(1 - p)$

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

Quotes

“Software systems go through a predictable lifecycle starting with small well-crafted solutions fully understood by a single person, through the rapid growth into a monolith of technical debt, thence fissioning into an ad hoc collection of fragile services, and finally into a well-engineered distributed system able to scale reliably in both breadth (more users) and depth (more features). It’s easy to see what needs to be done from the outside (make it more reliable!) and much harder to see the path from the inside.”

“Managing scale is not only about managing traffic volume—it also involves managing risk and availability. Often, all these things are different ways of describing the same problem, and they all go hand in hand. Thus, to properly discuss scale, we must also consider availability, risk management, and modern architecture paradigms such as microservices and cloud computing.”

Reliability & Availability

- › Key requirements for IT applications/systems
- › Reliability & availability
 - Unreliable applications suffer poor availability
 - Reliability and availability are different
 - *Reliability*
 - The ability of your system to perform the operations it is intended to perform without making a mistake.
 - *Availability*
 - The ability of your system to be operational when needed in order to perform those operations.

Availability

- › Measured by the proportion of the required time it is useable
 - 100% available during business hours?
 - No more than 2 hours scheduled downtime per week?
 - 99.99% availability (all day, every day)?
- › Loss of availability includes
 - Time to detect failure
 - Time to correct failure
 - Time to restart application & recover
 - Time for scheduled maintenance/upgrades if system is unavailable

$$\text{Site availability percentage} = \frac{\text{total_seconds_in_period} - \text{seconds_system_is_down}}{\text{total_seconds_in_period}}$$

Acceptable availability?

- › 3 nines is considered acceptable availability for basic Web applications
 - 43 minutes of downtime every month
- › 5 nines is acceptable for highly available Web application
 - 26 seconds of downtime every month
- › Includes all sources of unavailability
 - Scheduled maintenance during 'available' hours
 - Ops mishaps
 - Recovery/fail-overtimes

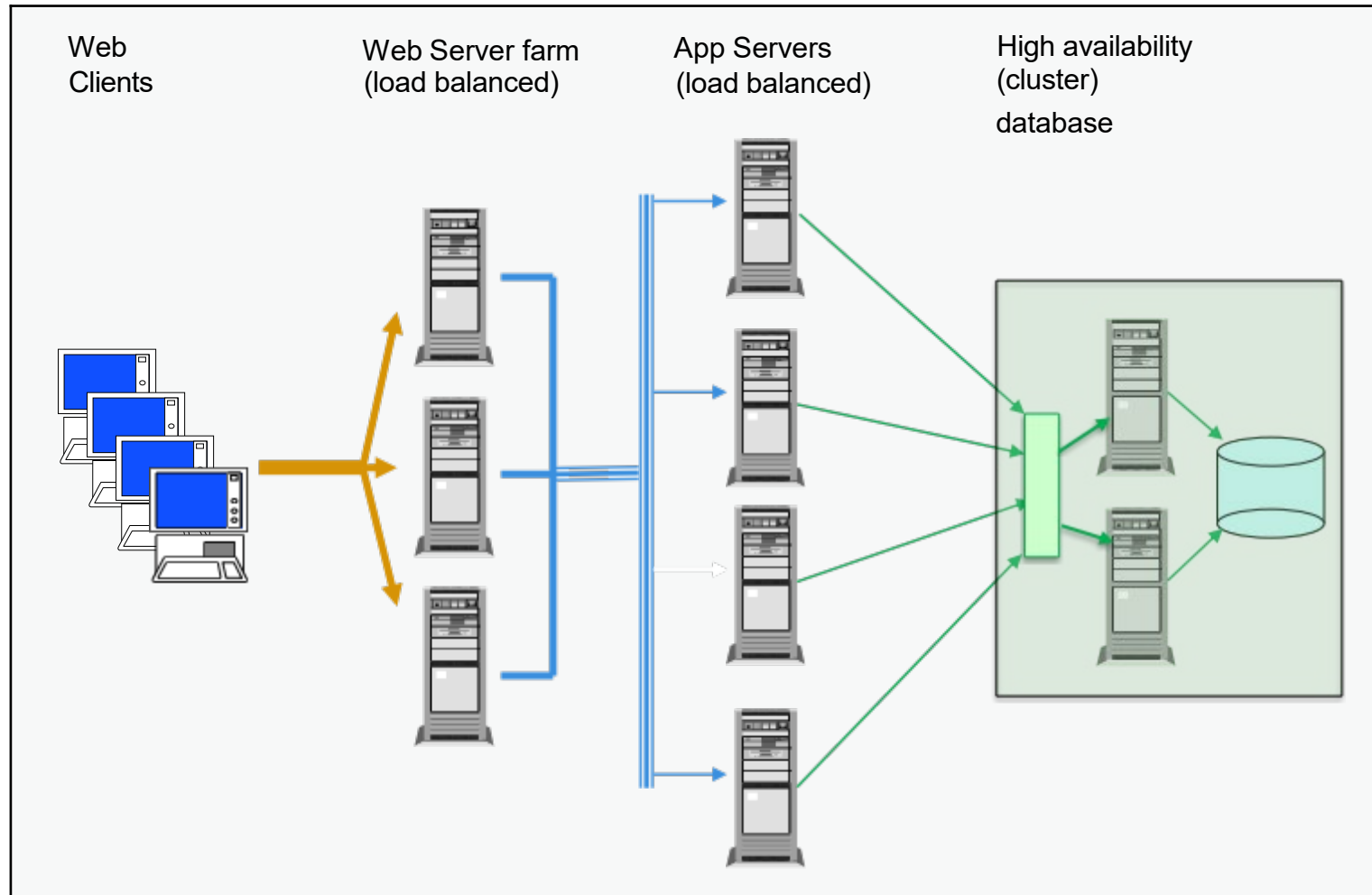
Nines	Percentage	Monthly outage ^a
2 Nines	99%	432 minutes
3 Nines	99.9%	43 minutes
4 Nines	99.99%	4 minutes
5 Nines	99.999%	26 seconds
6 Nines	99.9999%	2.6 seconds

^a This assumes a 30-day month with 43,200 minutes in the month.

Availability

- › Strategies for high availability
 - Eliminate single points of failure
 - Replication and failover
 - Automatic failure detection and restart
 - Planning for failure (risks & mitigations)
- › Redundancy is the key to availability
 - No single points of failure – not as simple as it sounds ...
 - Spare everything
 - Disks, disk channels, processors, power supplies, fans, memory, ..
 - Applications, databases, ...
 - Hot standby - quick changeover on failure (but costs more?)
 - Warm standby - slower changeover (but cheaper?)
 - Highly-available databases are a good topic for a long lecture ...

Simple available system



Availability

- › “Everything fails all the time” Werner Vogels, CTO of Amazon
 - Build with failure in mind
 - Always think about scaling
 - Find and mitigate risks
 - Monitor availability
 - Respond to availability issues in a predictable and defined way
- › Causes
 - Resource exhaustion
 - Unplanned load-based changes (and other hacks)
 - Increased number of moving parts (including people)
 - Outside dependencies
 - Technical debt (and complexity)

Availability

- › Often a question of application design and state management
 - Stateful vs stateless
 - What happens if a server fails?
 - Can requests go to any server?
 - Synchronous method calls or asynchronous messaging?
 - Reduce dependency between components
 - Failure tolerant designs
 - Services & decomposition
 - Strict borders between services
 - Contracts, SLAs, responsibility
 - Monitoring
 - Automation

Redundancy provides Availability

- › Passive or active standby systems
 - Re-route requests on failure
 - Server-side listener? Client-side?
 - Continuous service (almost)
 - Recover failed system while alternative handles workload
 - May be some hand-overtime (db recovery?)
 - Active standby reduce this (at 2x system cost...)
 - Redundant 'hot plug' hardware ('server-grade')
- › What happens to in-flight work?
 - State recovers by aborting in-flight ops & doing db recovery
 - Who is responsible for detecting failures & retrying?
 - Idempotent service requests?

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

Terminology: Fault and Failure

- › Any system or component has specified (desired) behaviors, and observed behaviors
- › Fault (defect): something gone/done wrongly
- › Failure: the observed behavior is different from the specified behavior
- › Once the fault occurs, there is a “latent failure”; sometime later the failure becomes effective (observed)
- › Once the failure is effective, it may be reported and eventually repaired;

What fails?

- › Anything can fail!
 - Dealing with a failure may cause more faults!
- › Example
 - Hardware
 - Software
 - OS, DBMS, individual process, language RTE, network stack, etc
 - Environment (power, cooling, network, ...)
 - Applications

“Everything fails all the time” ...

How does it fail?

- › Failstop
 - System ceases to do any work
- › Hard fault
 - Leads to: System continues but keeps doing wrong things
- › Intermittent (“soft”) fault
 - Leads to: Temporary wrong thing then correct behavior resumes
- › Timing failure
 - Correct values, but system responds too late

Repeatability?

- › If system is run the same way again, will the same failure behavior be observed?
 - Does re-running same data through same system result in the same error?
- › “Heisenbugs”
 - System does not fail the same way in different executions
 - May be due to race conditions, cosmic rays, power spikes, ...
 - Very common in practice
 - May not fail on almost all re-runs ...
- › “Bohrbugs”
 - Reproducible

Analyzing failure

- › Root cause analysis
- › Find the fault that led to failure
- › Then find why the fault was made
 - And why it wasn't detected/fixed earlier
- › See http://en.wikipedia.org/wiki/5_Whys

My car will not start. (the problem)

1. Why? - The battery is dead. (first why)
2. Why? - The alternator is not functioning. (second why)
3. Why? - The alternator belt has broken. (third why)
4. Why? - The alternator belt was well beyond its useful service life and has never been replaced. (fourth why)
5. Why? - I have not been maintaining my car according to the recommended service schedule. (fifth why, root cause)

Attacks or accident?

- › Some failures are due to malice: deliberate attempt to cause damage
- › Most are due to bad luck, incompetence, or choice to run risks
 - Often several of these together!

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

Memoryless fault model

- › Failure in each short period is independent of other failures in another short period
 - There is no effect from the lack of failure in the past
 - Contrast with gambler's fallacy
 - Long run of heads says nothing about getting a 'tail' next
 - Assuming the coin is not biased

Failure rate

- › Express the possibility of failure as AFR (annualized failure rate)
 - Expected ratio of failures to size of population, over a year of operation
 - How is it possible that $AFR > 1$?
- › Can then compute expected number of failures during other periods
 - Recall: approx 30,000,000 sec in 1 yr
 - Almost 10,000 hrs in 1 yr

Warning

- › AFR = 1% means: in 100,000 cases, 1000 is the expected (mean) number that fail during a year
 - It does not say: 1000 will definitely fail during the year
 - It is not necessarily saying: there is 50% chance that more than 1000 fail, and 50% chance that fewer fail
 - It does not say: my system will fail if I wait for 100 years

Bathtub fault model

- › Failure is more likely very early in system lifecycle, and very late in system lifecycle
 - Initially, many construction faults are discovered and fixed
 - Eventually parts wear out
 - Software also has “bit rot” due to shifting uses, other changes etc
- › Failure is rare in the extended middle of the system lifecycle

MTTF

- › Express current failure rate through mean-time-to-failure
 - $AFR = 1/MTTF$
 - $MTTF = 1/AFR$ years
 - This is often too long to be known through testing
- › Many vendors give “best case” MTTF
 - “system is guaranteed to perform like this or worse”

Disk failure rates

Annualized Hard Drive Failure Rates for Active Drives

Reporting Period: 4/20/2013 thru 12/31/2018 for drives models active as of 12/31/2018

MFR	Models	Drive Size	Maximum Drive Count	Total Drive Days	Total Failures	AFR
HGST	HMS5C4040ALE640	4 TB	8,570	10,625,825	152	0.52%
HGST	HMS5C4040BLE640	4TB	16,168	14,903,624	188	0.46%
HGST	HDS5C4040ALE630	4 TB	2,706	4,497,542	95	0.77%
HGST	HUH728080ALE600	8 TB	1,045	469,930	10	0.78%
HGST	HUH721212ALN604	12 TB	1,278	71,079	1	0.51%
Seagate	ST4000DM000	4 TB	34,744	45,272,013	3,441	2.77%
Seagate	ST6000DX000	6 TB	1,891	2,522,824	74	1.07%
Seagate	ST8000DM002	8 TB	9,891	8,215,991	226	1.00%
Seagate	ST8000NM0055	8 TB	14,396	7,930,051	214	0.98%
Seagate	ST10000NM0086	10 TB	1,220	569,367	7	0.45%
Seagate	ST12000NM0007	12 TB	31,146	8,132,638	310	1.39%
Toshiba	MD04ABA400V	4 TB	146	194,960	5	0.94%
Toshiba	MD04ABA500V	5 TB	45	62,730	2	1.16%
Toshiba	MG07ACA14TA	14 TB	1,205	108,536	9	3.03%
WDC	WD60EFRX	6 TB	458	654,360	71	3.96%
Totals:			124,909	104,231,470	4,805	1.68%

Repair

- › After a failure, system or component can be repaired/replaced
 - This takes time
 - And why server-class equipment has 'hot-plugging' capabilities & redundancy
- › MTTR is average time from failure till repair is completed (and system resumes working correctly)

Availability

- › What fraction of a time-period is system functioning correctly?
- › Availability = $MTTF / (MTTF + MTTR)$
- › Often quoted as “number of nines”
- › “3 nines” means 99.9% available
 - Downtime of approx 500 mins each year

Nines	Percentage	Monthly outage ^a
2 Nines	99%	432 minutes
3 Nines	99.9%	43 minutes
4 Nines	99.99%	4 minutes
5 Nines	99.999%	26 seconds
6 Nines	99.9999%	2.6 seconds

^a This assumes a 30-day month with 43,200 minutes in the month.

Parts and the whole

- › When a system is made up of several components
 - How does failure of a component impact the system as a whole?
- › Often failure of one component means overall failure
- › But some designs use redundancy
 - Allow system to keep working until several components all fail
 - Beware of hidden dependencies!
 - Application load-balanced across multiple physical servers (great!)
 - Servers are all in the same rack (whoops!)

Calculations for systems of components

- › For a system whose functioning requires every component to work
 - Assume that component failures are independent!
- › $AFR(\text{system}) = \text{Sum}(AFR \text{ of all components})$
- › $\text{Availability}(\text{system}) = \text{Product}(\text{availability of components})$
- › Examples in tonight's tutorial

Outline

- Performance
 - Approaches to Performance Prediction
 - Review of maths skills
 - Operational Laws
 - Performance bounds for closed systems
 - $M/M/1$ systems
- Availability
 - Types of Faults and Failures
 - Estimating reliability and availability
 - Techniques for dealing with failure

Availability focus

- › Focus #1: Build with Failure in Mind
 - Design patterns for handling failure, partial functionality, dependency failures
- › Focus #2: Always Think About Scaling
 - Scalable patterns, using additional resources, adding resources
- › Focus #3: Mitigate Risk
 - Risk analysis & (tested) mitigations
- › Focus #4: Monitor Availability
 - How do you know when things are broken, and what is broken?
- › Focus #5: Respond to Availability Issues in a Predictable and Defined Way
 - Automation and scripting rather than *ad hoc* 'fixes'

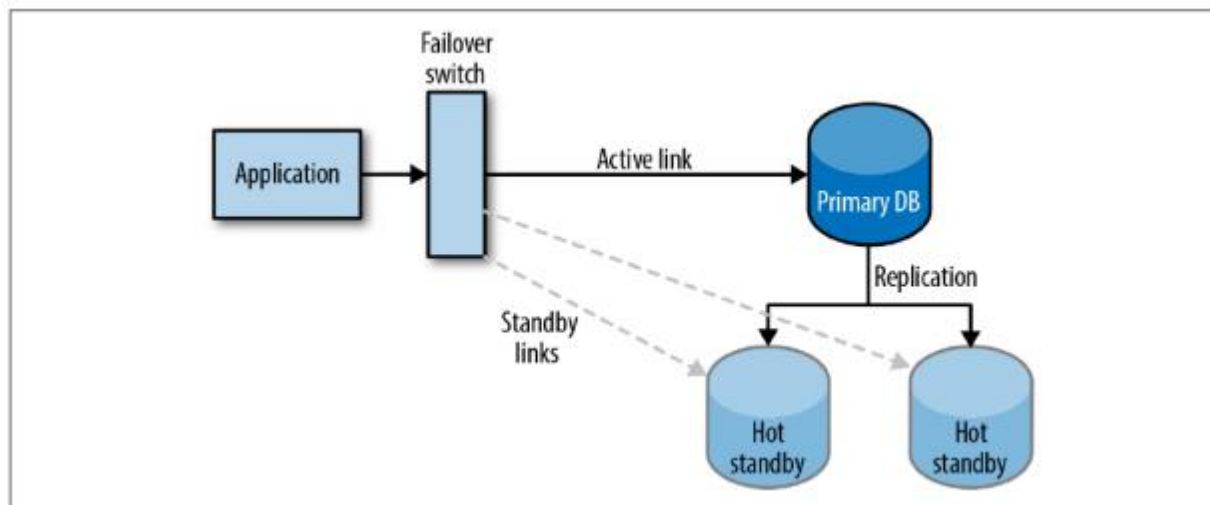
Risks

- › Knowing what can go wrong
 - You cannot possibly manage the risk in your system if you cannot identify the risk in your system (excluding “unknown unknowns”)
- › Full multi-team risk analysis
 - Including Likelihood, Severity and Mitigation
 - Live, regularly-reviewed document

	A	B	C	D	E	F	G	H	I	J	K	L	M
	Risk ID	System	Owner	Description	Date Identified	Likelihood	Severity	Mitigation Plan	Status	ETA	Monitoring	Triggered Plan (what to do if risk occurs?)	Comments
1	1	FrontEnd	FE Dev Team	Requires User Identity service to function. Front end fails if service is down.	10/13/15	Low	High	Perhaps we can cache the data for a period of time?	Open	3/26/16	Yes	If it happens often, we may have to look into improving independence on UI service.	
2													
3													
4													
5													
6													
7													
8													
9													
10													
11													
12													
13													
14													
15													
16													
17													
18													
19													
20													

Risk mitigation

- › Multiple, stateless applications running against a database
 - Database on server-grade hardware with RAID storage
 - Likelihood of failure is low, severity is very high
 - Single point of failure
- › Mitigation plan
 - Implement hot-standby database replicates



Testing mitigations

- › Mitigation plans are just pieces of paper unless they're tested ...
 - 'Game Days'
 - Injecting failures, testing mitigations
 - Test environment or production environment??
 - Different answers for different failures/mitigations
 - Testing on production environment OK if mitigations successful
 - How accurately does the test environment mimic production?
 - Scaling? Resources?
 - Can you really survive server, network, data centre failures?
 - Netflix 'Chaos Monkey'
 - Injecting errors into production system (during the working day)
 - Testing resilience and error recovery

Logging

- › Record what happens in a stable, independent place
 - Especially record all user inputs and responses
- › There are frameworks that make this somewhat easy
- › Essential to deal with mess left by failure

Styles of recovery

- › Backward: Restore system to a sensible previous state, then perform operations again
- › Forward: Make adjustments to weird/corrupted state, to bring it to what it ought to have been

Principles

› Idempotence

- Make sure that if you perform an operation again, it doesn't have different effects
 - Eg write(5) is idempotent, add(2) is not
 - Create idempotence by knowing what has already been done (store timestamps with the data)

› Causality

- Make sure that if you perform operation again, the system is in the same state it was in when op was done originally
 - Or at least, in a state with whatever context is necessary

Fail-fast components

- › Design of a larger system from redundant parts, is easier if the parts are failstop
 - And availability is best if they are quick to repair
- › So, don't try to keep going when a problem is observed
 - Report the error/problem unambiguously (if possible)
 - Timeouts for service calls
 - Callers need to avoid calling failed components
- › Wait for the repair or gracefully degrade service
 - Are some components/services 'nice' but not essential?
 - Can you return a static list of 'most popular' products if product search fails?
 - (part of mitigation strategy)

N-plex Redundancy

- › Have several (n) copies of the same component
- › When one fails, use another
 - And start repairing the first
- › So system works as long as enough components are working
- › Redundancy is also the key to reduced maintenance downtime
 - Rolling upgrades (take down service, upgrade, bring back on-line)
 - May need to over-provision to allow for failures during upgrade periods ...

Calculating failure rates

- › Assume that failures are independent
 - Not very realistic: eg environment can affect all in the same way
- › Availability(n-plex)
$$= 1 - (1 - \text{Avail}(\text{component}))^n$$
- › $\text{MTTF}(\text{n-plex}) = \text{MTTF}(\text{component})^n / (n * \text{MTTR}(\text{component})^{(n-1)})$
- › You can calculate MTTR from Availability and MTTF

Two mistakes high

- › How much redundancy is needed?
 - Service designed to handle 1,000 requests per second (req/sec).
 - Each single node in your service can handle 300 req/sec.
 - Question: How many nodes do you need to handle your traffic demands?
- › Some basic maths should come up with a good answer:

$$number_of_nodes_needed = \left\lceil \frac{number_of_requests}{requests_per_node} \right\rceil$$

$$number_of_nodes_needed = \left\lceil \frac{1000 \text{ req/sec}}{300 \text{ req/sec}} \right\rceil = \lceil 3.3 \rceil = 4 \text{ nodes}$$

- › Is 4 nodes the right answer?
 - No – need 5 nodes if you want to survive a single node failure
 - Always need 4 functioning nodes to avoid overloading
 - What if you want to do rolling upgrades?

Lab/Tutorial W8:
RESTful Web Services

Lecture W10 (Oct. 13):
Professional pathway towards
software architect



THE UNIVERSITY OF
SYDNEY



Further reading

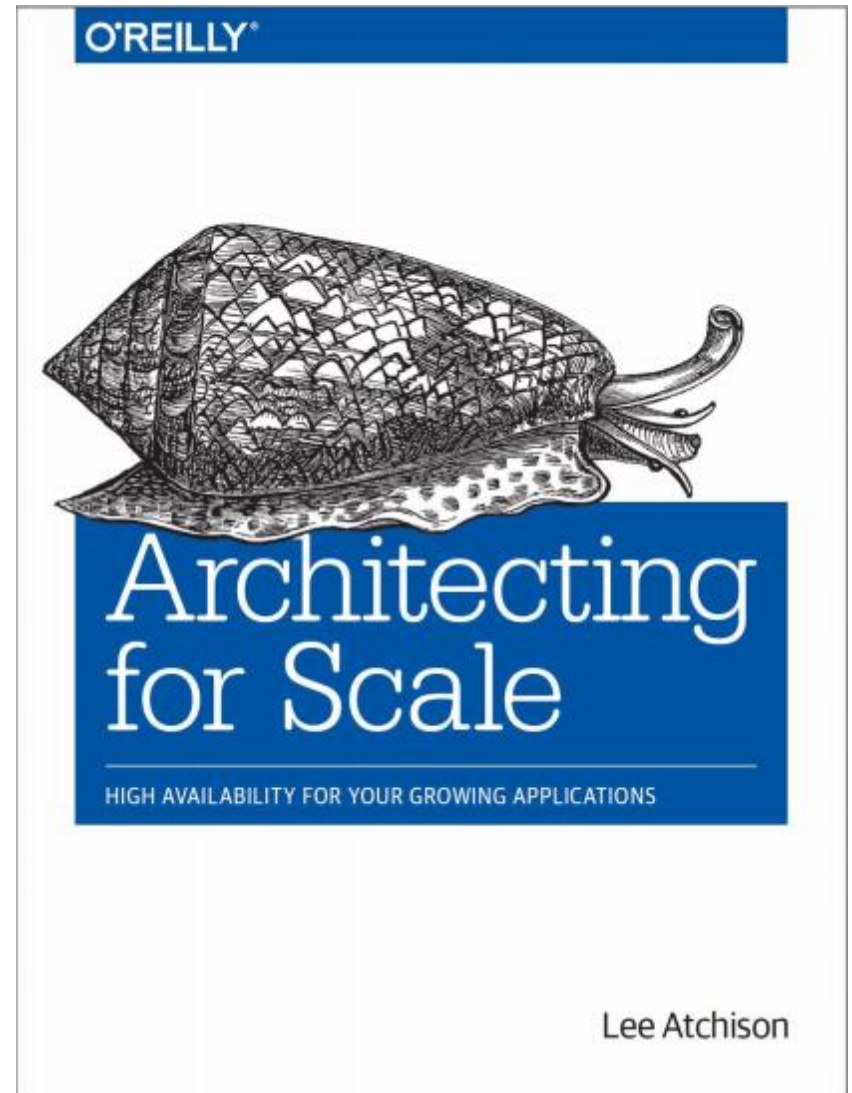
- › D. Menasce, V. Almeida, L. Dowdy “Performance by Design”, pub by Prentice-Hall, 2004
- › B. Schoeder, A. Wierman, M. Harchol-Balter “Open versus Closed: A Cautionary Tale”, from NSDI Conference, 2006
- › M. Harchol-Balter “Performance Modeling and Design of Computer Systems: Queueing Theory in Action”, Cambridge University Press, 2013

Further reading

Covers the architectural and organisational aspects of building high-availability systems

- Risk management & mitigation
- Services
- Redundancy & resilience

<http://shop.oreilly.com/product/0636920047070.do>



References

- › “REST in Practice”, by Jim Webber, Savas Parastatidis and Ian Robinson; O'Reilly 2010
- › An Introduction to RESTful Services With WCF
<http://msdn.microsoft.com/en-us/magazine/dd315413.aspx>
- › A Guide to Designing and Building RESTful Services with WCF3.5
<http://msdn.microsoft.com/en-us/library/dd203052.aspx>
- › More on REST <http://msdn.microsoft.com/en-us/magazine/dd942839.aspx>
- › Wikipedia article “Representational State Transfer”
http://en.wikipedia.org/wiki/Representational_State_Transfer
- › “Architectural Styles and the Design of Network-based Software Architectures” by R. Fielding (PhD thesis, 2000) <http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>